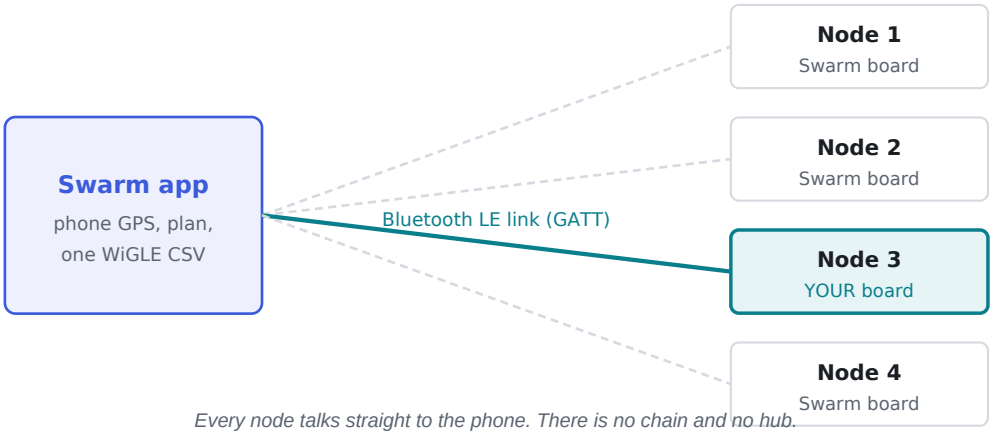
Adding Swarm node support to your firmware

A step-by-step guide to making an ESP32 wardriving board work as a node in a Swarm rig, using the Bluetooth LE link to the Swarm Android app.

What you are building

A Swarm rig is one Android phone running the Swarm app plus any number of small scanning boards (nodes). The phone is the boss: it numbers the boards, gives each one its share of the Wi-Fi channels, starts and stops the scan, and writes every sighting into one WiGLE CSV using its GPS. The boards do the radio work and keep what they find until the phone confirms it has it.



Every node talks straight to the phone. There is no chain and no hub.

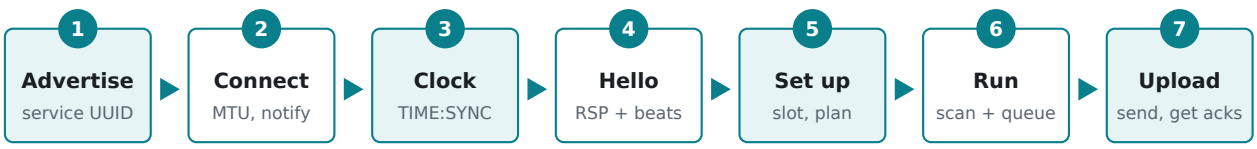
To become a node, your firmware keeps doing its own scanning and adds a Bluetooth LE **peripheral** that speaks a short text-line protocol. Your board has to:

- **Advertise** the Swarm service and expose three GATT characteristics.
- **Answer the hello**, then send a **heartbeat** line every 3 seconds.
- **Follow orders**: store its node number and channel plan, start and stop scanning when told.
- **Queue every sighting with a sequence number** and keep it until the phone acknowledges it, so nothing is lost while the phone is out of range.

Optional extras: the owner lock (the board then works only for its owner's phone), and reporting a fix from your own GPS.

> **The one rule that trips everyone up**
> The app only keeps boards that answer the hello **and** send a heartbeat (`DATA:BEAT`) within 4 seconds of it. A board that connects but never sends a heartbeat is never shown, however well the rest works.

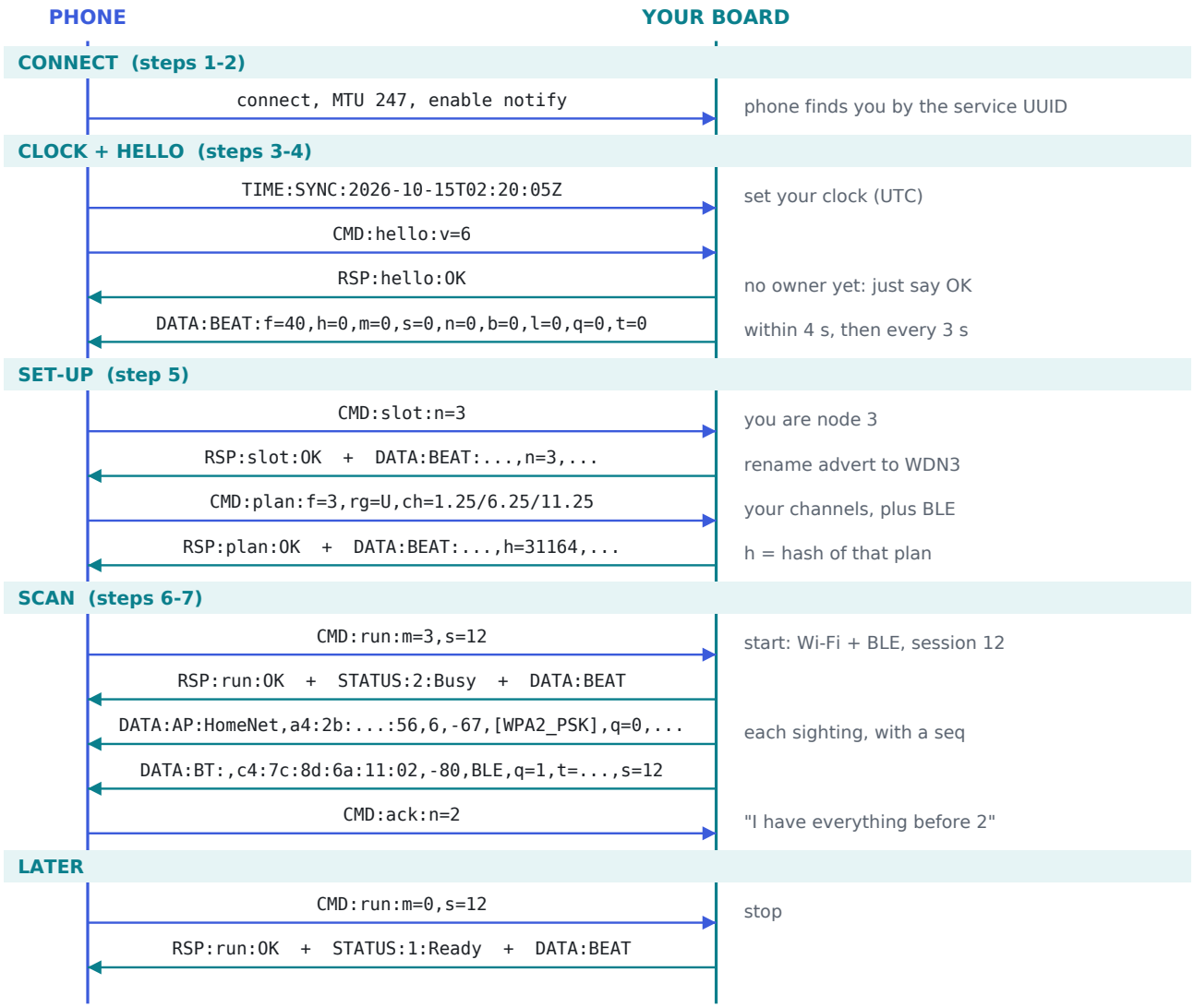
The whole process at a glance



Steps 1 to 7 below follow this order. Each step says what the phone does, what your board must do, and how to tell it worked.

What a link looks like

Here is a complete first meeting between the phone and a brand-new board with no owner, then a scan. Read it once before the details: every later section explains one part of it.



Blue arrows are lines the phone writes to your Command characteristic; teal arrows are lines you notify on Response. Every command except `ack` gets an `RSP` line and then a fresh heartbeat. The plan hash shown is the real one for that plan (see Reference).

Step 1. Bluetooth setup

Advertise connectably with the service UUID in the advert: the phone finds boards by that UUID, not by name. Name the board `WDU` until it has a node number, and `WDN<number>` after (for example `WDN3`). The name is only for people, but keep the pattern.

	UUID	Properties	Use
Service	`4fafc201-1fb5-459e-8fcc-c5c9c331914b`	advertised	
Command	`beb5483e-36e1-4688-b7f5-ea07361b26a8`	write, write without response	phone to board
Response	`beb5483e-36e1-4688-b7f5-ea07361b26a9`	notify	board to phone
Status	`beb5483e-36e1-4688-b7f5-ea07361b26aa`	read	JSON, e.g. `{"status":1,"battery":-1,"build":3}`

Lines

Every message in both directions is one text line: `TYPE:SUBTYPE:PAYLOAD` ending in `\n`. Keep the second colon even when the payload is empty (`CMD:ping:`). Arguments are `key=value` pairs separated by commas.

Line	Direction	Meaning
`CMD:<name>:<args>`	phone → board	a command
`RSP:<name>:OK`	board → phone	command done; `RSP:<name>:ERROR[:reason]` if not
`STATUS:1:Ready` / `STATUS:2:Busy`	board → phone	idle / scanning
`DATA:<kind>:<fields>`	board → phone	a sighting, a heartbeat or a GPS fix
`TIME:REQUEST:`	board → phone	every 5 s until your clock is set
`TIME:SYNC:2026-10-15T02:20:05Z`	phone → board	the time, UTC, ISO-8601

Packing rules

- Send nothing on Response until the phone has enabled notifications.
- One notify may carry several lines, and a long line may span several notifies: the phone joins the stream and splits it on `\n`. Keep each notify within MTU minus 3 (the phone asks for MTU 247, so 244 bytes).
- Keep each line you send under 160 bytes. Accept command writes up to 512 bytes: a long plan arrives as a long write, and one write may hold several lines. Handle them in order.
- In SSIDs and names, replace commas, CR, LF and NUL with _ and cut them to 24 bytes.

It worked when: the phone shows your board under nearby Bluetooth devices with the service, and the Swarm app connects to it after you tap FIND MY NODES (it will drop again until step 4 is done).

Step 2. Connecting

The phone connects, asks for MTU 247 and enables notifications on Response. Then it writes, in this order:

1. `TIME:SYNC:<UTC time>`: the clock (step 3).
2. `CMD:ack:n=<seq>`: only if it already knows your board, so you can drop what it already has.
3. `CMD:hello:v=6`: the hello (step 4).

Be ready to receive the first write as soon as notifications are on. After a disconnect, start advertising again straight away: the phone reconnects on its own, and it may disconnect you on purpose (see

Things the phone does).

Step 3. The clock

Set your clock from `TIME:SYNC` (seconds are enough). Until it is set, send `TIME:REQUEST:` every 5 seconds and keep flag 32 clear in your heartbeat; once set, add flag 32.

> **Why the clock matters**
> The phone places each sighting on the map by looking up its own GPS track at the sighting's time `t`. A sighting with `t=0` or a wrong clock can't be placed and is dropped from the log. Hold sightings until the clock is set, or fix up their times once it is.

Step 4. Hello and heartbeat

Answer `CMD:hello:v=6` with `RSP:hello:OK` and start heartbeats at once. (If you add the owner lock, an owned board answers with a challenge instead: see step 8.) From then on, send a heartbeat every 3 seconds and right after every command you accept, except `ack`.

The heartbeat is how your board reports its state. The phone compares it with what it wants and sends only the commands that differ, so a wrong field means the phone keeps repeating a command.

```
DATA:BEAT:f=<flags>,h=<plan hash>,m=<run>,s=<session>,n=<node>,
          b=<backlog>,l=<lost>,q=<next seq>,t=<oldest seq>,v=<build>,rg=<E|U>
(one line; wrapped here to fit the page)
```

Field	What to put in it
`f`	flags, added together (table below)
`h`	hash of the plan you hold, in decimal (Reference); 0 before you have one
`m`	run mask you are obeying: 1 Wi-Fi, 2 BLE, 3 both, 0 stopped
`s`	session number from the last `run`
`n`	your node number, 0 = none yet
`b`	sightings queued and not yet acknowledged
`l`	sightings dropped because your queue overflowed
`q`	the seq your next sighting will get
`t`	the oldest seq still in your queue (lets the phone skip a gap after a reboot)
`v`	your firmware build number (optional, shown in the app)
`rg`	region you hold, `E` or `U`, from the last plan (optional but recommended)
`bv, tc, up, hp`	optional health: battery mV (0 = unknown), chip temperature °C (-128 = unknown), minutes since boot, free RAM in KB

Flag	Meaning
1	scanning right now
4	you have your own GPS **and** hold a fix right now (step 9)
8	your radio can scan 5 GHz (without it the phone never gives you 5 GHz channels)
16	your queue spills to flash
32	clock set by the phone since boot
64	you have an owner (owner lock)

Flag	Meaning
2, 128 and up	reserved: keep them clear (128 marks a different kind of node)

Example: `DATA:BEAT:f=105,h=8195,m=3,s=12,n=2,b=40,l=0,q=1873,t=1833,v=3,rg=U` (105 = 64 + 32 + 8 + 1: owned, clock set, 5 GHz, scanning).

It worked when: your board appears in the app's SWARM tab with a node number and ONLINE.

Step 5. Commands from the phone

Answer every command except `ack` with an `RSP` line, then send a heartbeat. An unknown command gets `RSP:<name>:ERROR:unknown command`.

Command	What to do	Answer
`CMD:slot:n=<node>`	Store the node number in flash; advertise as `WDN<node>`.	`RSP:slot:OK`
`CMD:plan:f=<flags>,rg=<E|U>,ch=<ch>.<dwell>/...`	Store the plan in flash exactly as sent (details below).	`RSP:plan:OK`
`CMD:run:m=<mask>,s=<session>`	Start or stop scanning. `m`: 1 Wi-Fi, 2 BLE, 3 both, 0 stop. Keep it in flash so a reboot or a lost phone doesn't stop the scan. Stamp new sightings with session `s`.	`RSP:run:OK`, then `STATUS:2:Busy` or `STATUS:1:Ready`
`CMD:ack:n=<seq>`	The phone has every sighting before `seq`: delete them.	nothing, no heartbeat
`CMD:ping:`	Nothing.	`RSP:ping:OK`
`CMD:own:k=<32 hex>`	Owner lock (step 8).	`RSP:own:OK` or `RSP:own:ERROR`
`CMD:disown:`	Owner lock: forget the owner.	`RSP:disown:OK`

The plan

The phone spreads the channels over all nodes, so following the plan is what stops nodes scanning the same channels. Example: `CMD:plan:f=3,rg=U,ch=1.25/6.25/11.25`

- `f` = what to scan: 1 Wi-Fi, 2 BLE, 3 both.
- `rg` = region: `E` Europe, `U` USA. Set your radio's country to match and skip channels the region doesn't have (EU: 1-13, 36-64, 100-140. US: 1-11, 36-64, 100-144, 149-177). It is not part of the hash.
- `ch` = the Wi-Fi channels to hop through, in order. Each entry is `channel.dwell` with dwell in 10 ms steps: `6.25` = listen on channel 6 for 250 ms. Up to 48 entries.
- Scan BLE alongside when `f` includes 2.

> **If your firmware can't follow a channel list**
> Store the plan anyway and report its hash in the heartbeat, then scan what you can. If the hash doesn't match, the phone sends the plan again after every heartbeat and shows SYNCING PLAN.

Keep scanning without the phone

Once told to run, keep scanning and queueing until told to stop, even if the link drops or the board reboots. The phone collects the backlog when it is back in range. This is the heart of how Swarm works at speed: boards drop in and out of range all the time.

Step 6. Sightings

Every sighting is one line, with three Swarm fields on the end:

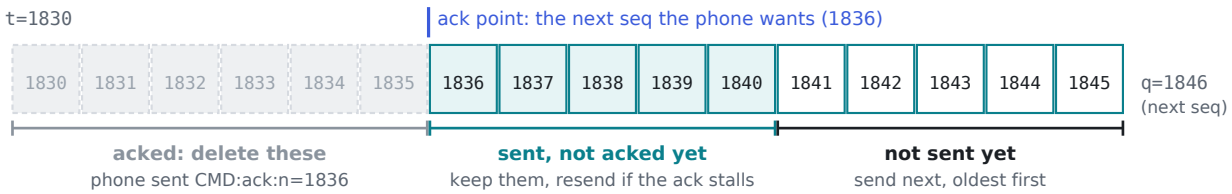
```
DATA:AP:<SSID>,<BSSID>,<channel>,<RSSI>,[<security>],q=<seq>,t=<unix time>,s=<session>
DATA:BT:<name>,<MAC>,<RSSI>,BLE,q=<seq>,t=<unix time>,s=<session>
```

- `security` is one of `OPEN`, `WEP`, `WPA_PSK`, `WPA2_PSK`, `WPA_WPA2_PSK`, `WPA2_ENTERPRISE`, `WPA3_PSK`, `WPA2_WPA3_PSK`, `UNKNOWN`, in square brackets.
- MACs as `aa:bb:cc:dd:ee:ff`. An empty SSID or name stays empty.
- `q` = the sighting's sequence number, `t` = unix second when you saw it (from the clock the phone set), `s` = session from the last `run`.
- Don't queue the same thing over and over: Swarm boards queue a given access point again only after 5 minutes and a given BLE device after 30 seconds. The phone keeps one row per device per drive anyway.

Example: `DATA:AP:HomeNet,a4:2b:b0:12:34:56,6,-67,[WPA2_PSK],q=1873,t=1792000805,s=12`

Step 7. The queue and the upload

This is what lets a board lose the phone for minutes and still hand over everything later.



- **Sequence numbers.** Give each queued sighting the next seq, counting up by one, and never reuse one, even across reboots. A simple way: reserve a block of seqs in flash and skip ahead to the next block after a reboot. Report the oldest one you still hold as `t` in the heartbeat so the phone stops waiting for the gap.
- **Keep** every sighting until it is acknowledged, in RAM and, if you can, spilling to flash. If the queue fills, drop the oldest and count them in `l`.
- **Upload.** After each hello, send from the oldest unacknowledged sighting, oldest first, as fast as the link allows. New sightings join the end of the queue.
- **Acks.** The phone answers `CMD:ack:n=<seq>` (at most every half second) meaning "I have everything before seq": delete those.
- **Stalls.** If you have sent everything and the ack hasn't moved for about 10 seconds, start again from the oldest unacknowledged one. The phone drops repeats by seq, so sending one twice is harmless.

It worked when: start a scan, walk the board out of range for a minute and back. Its backlog in the SWARM tab climbs while it is away and falls back to zero after it reconnects.

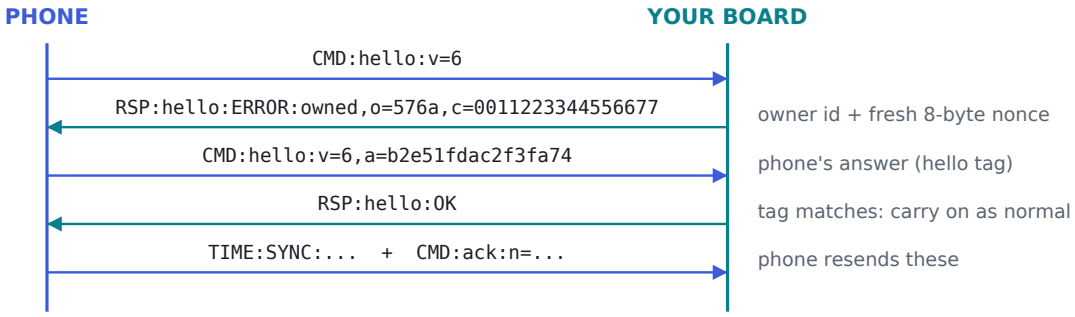
Step 8 (optional). The owner lock

The owner lock makes a board work only for its owner's phone, so a Swarm rig parked next to yours can't take your nodes. If you skip it, never set flag 64 and answer `CMD:own` with `RSP:own:ERROR`: your board then works with any Swarm phone that finds it.

Taking an owner

The first heartbeat without flag 64 makes the phone send `CMD:own:k=<32 hex chars>`. Store those 16 bytes in flash as the owner key, set flag 64 in your heartbeats, and answer `RSP:own:OK`.

Proving the owner on every later link




```
owner id  = HMAC-SHA256(key, "swarm-owner"), first 2 bytes, as a little-endian u16 (4 hex)
hello tag = HMAC-SHA256(key, "hello" + nonce[8] + address[6]), first 8 bytes (16 hex)
```

- `key` is the 16 raw bytes, not the hex text. The message is the five ASCII letters, then the nonce, then the address, with nothing in between.
- `nonce` is 8 fresh random bytes for every challenge. A wrong tag gets a new challenge.
- `address` is your board's Bluetooth address as the phone sees it, in printed order: `AA:BB:CC:DD:EE:FF` is AA BB CC DD EE FF. ESP-IDF and NimBLE store addresses reversed, so flip them.
- All hex is lower case on the way out; accept either case on the way in.
- Until the tag is proved, accept only `hello` and `ping`. Answer other commands with `RSP:<name>:ERROR:owned` (ignore acks and TIME:SYNC silently), send no sightings or heartbeats, and drop the link after 15 seconds.

Freeing a board

Give users a way to forget the owner so the board becomes new again. Swarm boards do it on a 5-second BOOT button hold, the serial command `forget`, or `CMD:disown`: from the owner's proven link (answer `RSP:disown:OK`).

Check your HMAC code against the test vectors in the Reference before testing with a phone.

Step 9 (optional). Your own GPS

If your board has a GPS, the phone can place your sightings by your fix instead of the phone's, and can start a scan on your fix when the phone has none. Needs Swarm app build 17 or later.

- Set flag 4 in the heartbeat **only while** you hold a fix. Clear it the moment the fix is lost; the phone falls back to its own GPS.
- While you have a fix, send about once a second: `DATA:GPS:lat=<deg>,lon=<deg>,acc=<m>,t=<unix s>[,alt=<m>]`
- `t` is the fix time, UTC (decimals are fine). `acc` is the horizontal accuracy in metres; the phone ignores fixes worse than 60 m. The short form `lat,lon,acc,t` is also accepted.

Example: `DATA:GPS:lat=51.500700,lon=-0.124600,acc=4.5,t=1792000805,alt=12`

It worked when: the board's line in the SWARM tab says OWN GPS.

Things the phone does that surprise people

What you see	Why, and what to do
The phone disconnects your board every so often during a scan	On purpose. Many boards can't scan BLE while a link is up, so when BLE is in the plan and your backlog is small the phone lets your board go for about 30 seconds to scan alone, then reconnects to collect. Keep scanning, advertise again at once, and resume from the oldest unacked sighting after the next hello.
The same command arrives again and again	Your heartbeat disagrees with what the phone wants (plan hash, node number, run mask, session or region). Fix the heartbeat field, not the command handler.
The board connects, then drops after about 4 seconds	No heartbeat arrived within 4 seconds of the hello.
Sightings arrive but never reach the CSV	Their `t` is 0 or wrong (clock not set), or the phone had no GPS fix at that time.
The backlog never falls	Acks aren't being applied, or seqs restart after a reboot. Check that you delete everything before the ack and that `t` in the heartbeat moves forward.
5 GHz channels never show up in your plan	Flag 8 is missing from your heartbeat.
Your board is listed as BASIC	It answered the hello but sent no heartbeat. The app then drives it with whole-band commands and live sightings only: no plan, no queue, nothing kept while out of range.

Testing with the app

1. Power the board, open Swarm and tap **FIND MY NODES** on the BASIC tab.
2. Within a few seconds it shows up on the SWARM tab as the next node number, ONLINE. SYNCING PLAN goes away once your heartbeat's `h` matches the plan it was sent.
3. Tap **START** on BASIC. Your board gets `CMD:run:m=3,...`, starts scanning, and its sightings appear in the live feed (ADVANCED, DRIVE).
4. Walk the board out of range and back. Its queued sightings upload and the backlog falls to zero.
5. Tap **STOP**. The board gets `CMD:run:m=0,...` and answers `STATUS:1:Ready`.

A USB serial log of every line you send and receive makes all of this much quicker to debug.

Checklist

- Advertises the service UUID, connectable, named `WDU` / `WDN<node>`.
- Command, Response and Status characteristics; incoming writes split on `\n`.
- `TIME:SYNC` handled; `TIME:REQUEST:` every 5 s until set.
- `CMD:hello:v=6` answered, heartbeats every 3 s and after each command.
- `slot`, `plan`, `run`, `ack`, `ping` handled; node number, plan and run state kept in flash.
- Sightings queued with seqs; sent with `q`, `t`, `s`; deleted on ack; resent after a hello or a stalled ack.
- Optional: owner lock, `disown` and a way to forget the owner; own GPS.
- Plan hash and HMAC checked against the test vectors.

Reference

Plan hash

32-bit FNV-1a over single bytes (`flags & 3`, the entry count, then each channel and its dwell), folded to 16 bits. Dwell is the number from the plan line (10 ms steps, 2 to 255). `rg` is not included. Send it in decimal.

```
uint16_t plan_hash(uint8_t flags, const uint8_t *ch, const uint8_t *dwell, uint8_t n) {
  uint32_t h = 2166136261u;
  #define MIX(b) h = (h ^ (uint8_t)(b)) * 16777619u
  MIX(flags & 3);
  MIX(n);
  for (uint8_t i = 0; i < n; i++) { MIX(ch[i]); MIX(dwell[i]); }
  #undef MIX
  return (uint16_t)((h >> 16) ^ (h & 0xFFFF));
}
```

Test vectors

Input	Expected
Plan `CMD:plan:f=1,ch=1.25/6.25/11.25`	`h=8195`
Owner key `000102030405060708090a0b0c0d0e0f`: owner id	`576a`
Same key, nonce `0011223344556677`, address `AA:BB:CC:DD:EE:FF`: hello tag	`b2e51fdac2f3fa74`

Numbers to match

Constant	Value
Protocol version in hello	6
Heartbeat interval	3 s, and after every accepted command except ack
Time request interval (until the clock is set)	5 s
Phone waits for the first heartbeat	4 s after the hello
Unproven (owned) link dropped after	15 s
MTU the phone asks for / usable per notify	247 / 244 bytes
Longest line sent / longest command write accepted	160 / 512 bytes
SSID or name kept	24 bytes
Plan entries	up to 48
Ack rate from the phone	at most every 0.5 s
Resend from the oldest unacked if the ack stalls	about 10 s
Phone's BLE turn (board unlinked to scan BLE alone)	about 30 s
Own-GPS fix: send rate / worst accuracy used	about 1 s / 60 m
Region channels, EU	2.4 GHz 1-13; 5 GHz 36-64, 100-140
Region channels, US	2.4 GHz 1-11; 5 GHz 36-64, 100-144, 149-177

This guide covers everything a node needs to join a rig.